SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Modal

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

Assessment Tool Weightage: 30%

QUESTIONS: **Total Marks:25**

Q1. Transfer Learning

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

Q2. DenseNet and PixelNet

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

Q3. Bidirectional RNN and Encoder–Decoder

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

Q4. BPTT and LSTM

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

Q5. Integrated Deep Learning Model

Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture. Justify the role of each component and explain how the complete system generates meaningful captions.

1. Transfer Learning – Plant Disease Detection

Problem Understanding:
Plant diseases can be detected automatically from leaf images using deep learning. Training a CNN from scratch requires a large dataset and high computational power. Transfer learning solves this problem by using a CNN pre-trained on a large dataset such as ImageNet.

Proposed Architecture:

```
Input Leaf Image → Data Augmentation → Pre-trained CNN → Global Average Pooling → Dense Layer → Softmax → Disease Class
```

A model such as **ResNet50, MobileNetV2, or EfficientNet** can be used. Initially, the convolutional layers are frozen because they already contain useful features such as edges, textures, shapes, and patterns. The final classification layers are replaced according to the number of plant disease classes.

Freezing and Fine-Tuning:

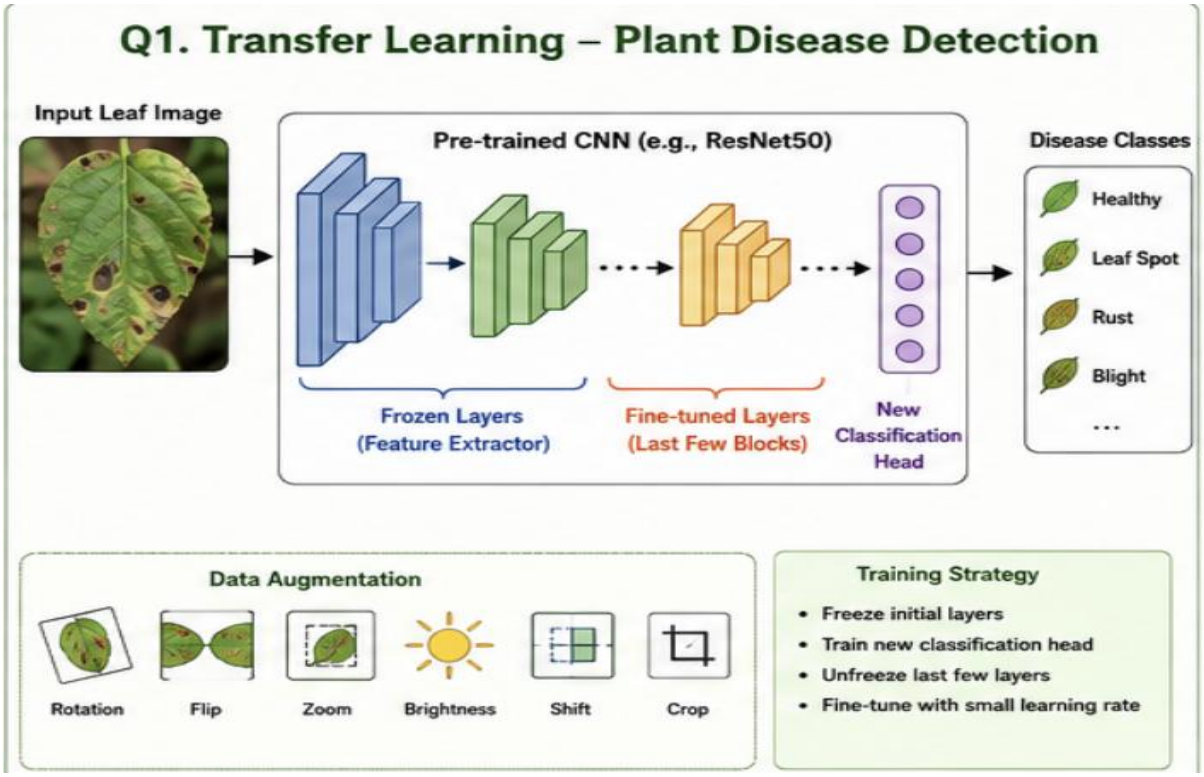
- Freeze the initial layers because they learn general features.
- Freeze most middle layers during initial training.
- Replace and train the final classification layer.
- After obtaining stable performance, unfreeze the last few convolutional blocks.
- Fine-tune these layers using a small learning rate.

Data Augmentation:

- Rotation
- Horizontal and vertical flipping
- Zooming
- Width/height shifting
- Brightness adjustment
- Random cropping
- Small changes in contrast

These techniques increase dataset diversity and reduce overfitting.

Expected Result:
The transfer learning model can accurately classify diseases such as **leaf spot, rust, blight, and healthy leaves** while requiring less training time than a CNN trained from scratch.



2. DenseNet and PixelNet – Semantic Segmentation

Problem Understanding:
Autonomous vehicles need to identify every pixel in a road scene, including roads, vehicles, pedestrians, buildings, traffic signs, and vegetation. Semantic segmentation assigns a class label to every pixel.

Proposed Architecture:

Input Image → DenseNet Feature Extractor → Dense Connectivity → Feature Maps → PixelNet Decoder → Pixel-wise Classification → Segmentation Map

DenseNet:
DenseNet connects each layer with all subsequent layers within a dense block. Therefore, features learned by earlier layers are reused by later layers.

For example:

Layer 1 → Layer 2 → Layer 3 → Layer 4

Each layer receives the feature maps produced by previous layers.

This provides:

- Better feature reuse
- Improved gradient flow
- Reduced vanishing-gradient problem
- Fewer parameters
- Better extraction of fine image features

PixelNet Concept:
PixelNet performs pixel-level prediction. Instead of predicting only one class for an entire image, it predicts the class of every pixel.

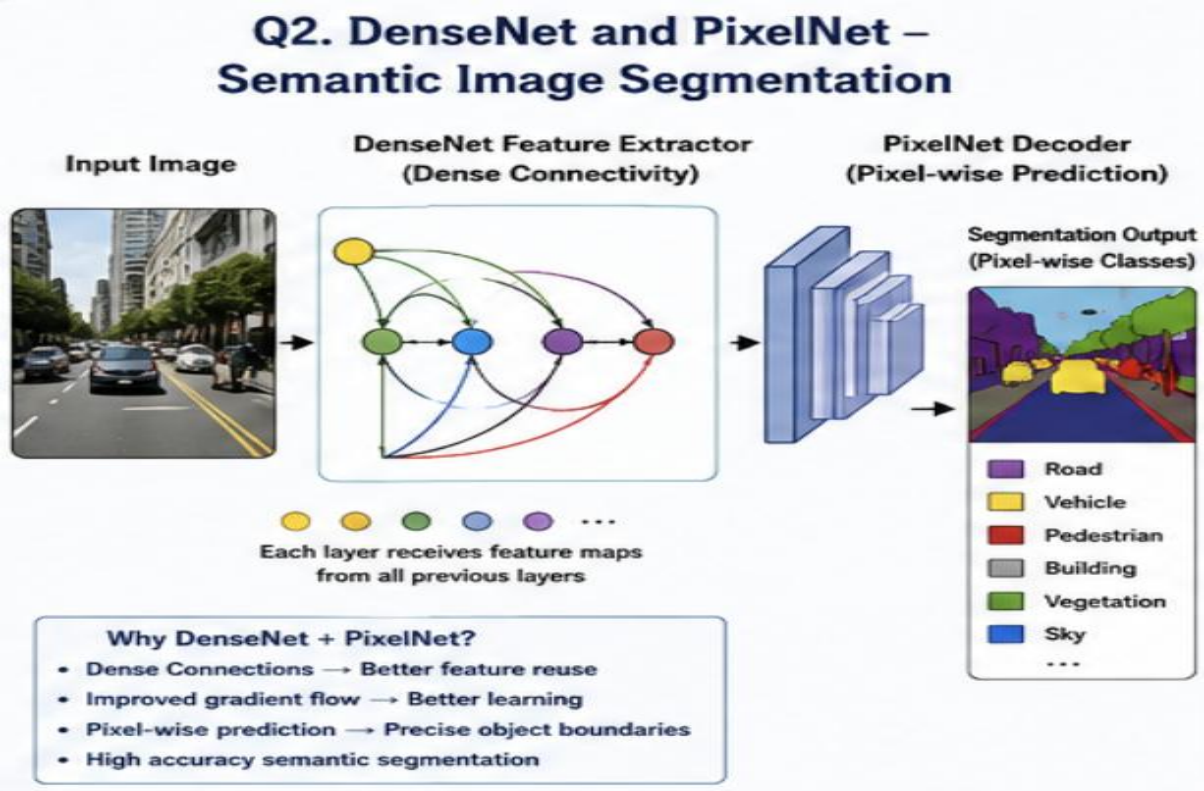
For example:

Road → Road pixels
Car → Car pixels
Pedestrian → Pedestrian pixels

Combined Architecture:

DenseNet acts as the **feature extractor**, while PixelNet performs **pixel-wise classification**. The combination preserves detailed spatial information and improves segmentation accuracy.

Expected Result:
The system produces a segmentation map in which each pixel is assigned to classes such as road, vehicle, pedestrian, building, and sky. This helps autonomous vehicles understand their surroundings accurately.



3.Bidirectional RNN and Encoder–Decoder – Machine Translation

Problem Understanding:
Machine translation converts a sentence from one language to another. The meaning of a word often depends on both the previous and following words. Therefore, a Bidirectional RNN can provide better contextual information.

Architecture:

```
Source Sentence
↓
Embedding Layer
↓
Bidirectional RNN Encoder
↓
Context Representation
↓
RNN/LSTM Decoder
↓
Target Sentence
```

Example:

```
English: I am going to school

↓

French: Je vais à l'école
```

Bidirectional RNN Encoder:
A normal RNN processes information in one direction:

```
Past → Present
```

A Bidirectional RNN processes the sequence in two directions:

```
Past → Present
Future → Present
```

The forward RNN captures previous-word information, while the backward RNN captures future-word information.

The final representation is:

h_t = [h_t forward ; h_t backward]

This provides richer contextual information.

Encoder:
The encoder reads the complete source sentence and converts it into contextual feature representations.

Decoder:
The decoder generates the translated sentence one word at a time.

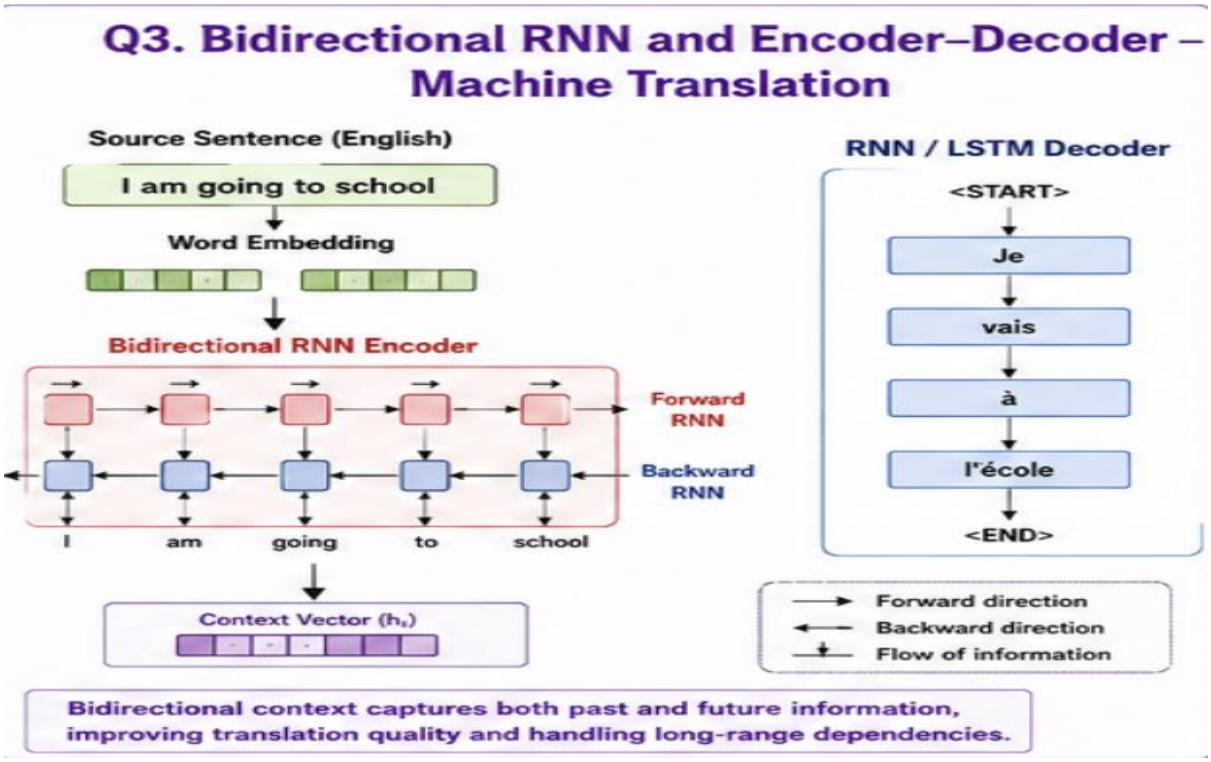
```
<START> → Je → vais → à → l'école → <END>
```

Advantages:

- Uses both past and future context.
- Better understanding of sentence structure.
- Improves translation accuracy.

- Handles long and complex sentences better.
- Useful for multilingual translation.

Expected Result:
The encoder–decoder model generates grammatically meaningful translations by using contextual information from the entire input sentence.



4. BPTT and LSTM – Stock Market Trend Prediction

Problem Understanding:
Stock prices are time-series data where previous values can influence future trends. A deep learning model can learn patterns from historical data and predict the future movement.

Input Features:

- Opening price
- Closing price
- Highest price
- Lowest price
- Trading volume
- Moving averages

Architecture:

Historical Stock Data → Normalization → Time-Series Sequences → LSTM → Dense Layer → Trend Prediction

For example:

Previous 30 days → LSTM → Next-day prediction

The output can be:

0 → Downward trend
1 → Upward trend

or a continuous predicted price.

BPTT

Backpropagation Through Time (BPTT) is used to train recurrent neural networks. The network is unrolled across time steps and the prediction error is propagated backward through these time steps.

t1 → t2 → t3 → t4 → Prediction

The error travels:

Prediction → t4 → t3 → t2 → t1

The weights are then updated using gradient descent.

Why LSTM instead of Standard RNN?

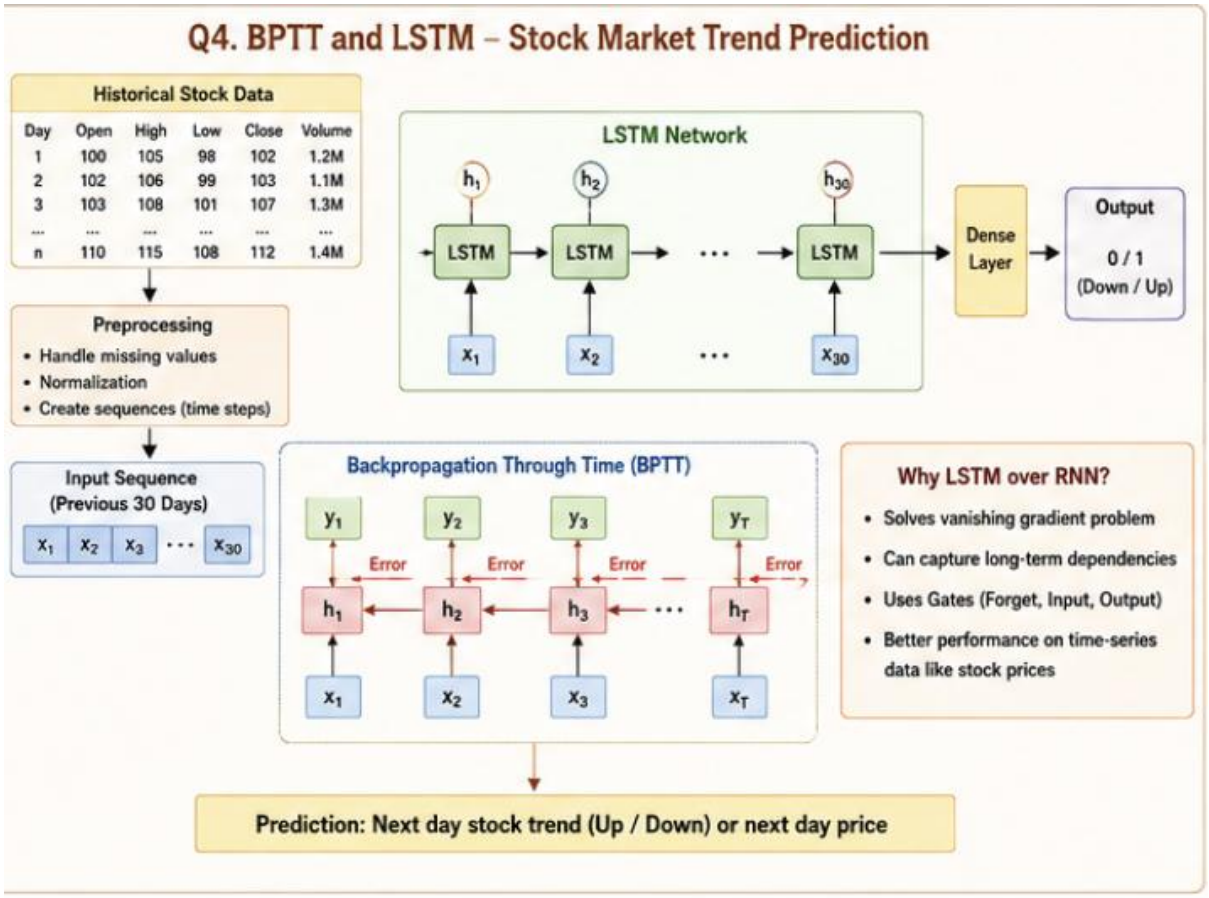
Standard RNNs suffer from the **vanishing-gradient problem**, especially for long sequences. LSTM solves this using a memory cell and three gates:

1. **Forget Gate** – decides what information to remove.
2. **Input Gate** – decides what new information to store.
3. **Output Gate** – decides what information to output.

Therefore, LSTM can remember important historical information for a longer period.

Expected Result:
The LSTM model learns temporal dependencies in historical stock data and predicts whether the next market movement is likely to be upward or downward.

Important: Stock prediction is inherently uncertain; the model provides a learned prediction, not a guaranteed future market result.



5. Integrated Deep Learning Model – Automatic Video Caption Generation

Problem Understanding:
Video caption generation automatically converts video content into a meaningful natural-language sentence. The system must understand both **visual information** and **temporal information**.

Integrated Architecture:

Input Video
↓
Frame Extraction
↓
Pre-trained CNN
↓
Visual Feature Extraction
↓
LSTM Encoder
↓
LSTM Decoder
↓
Word Prediction
↓
Generated Caption

1. Transfer Learning

A pre-trained CNN such as **ResNet, VGG, or EfficientNet** is used to extract visual features from video frames.

Example:

Video Frame → CNN → Feature Vector

The CNN is already trained on a large image dataset, so it can recognize useful visual patterns without training the complete network from scratch.

2. Feature Extraction

Frames are sampled from the video:

Frame 1 → Feature 1
Frame 2 → Feature 2
Frame 3 → Feature 3
...

These features represent objects, scenes, and visual information in the video.

3. LSTM Encoder

The LSTM encoder processes the sequence of visual features and learns temporal information.

For example:

Person → picks up → ball → throws ball

The LSTM understands the order of these actions.

4. LSTM Decoder

The decoder generates words sequentially.

<START>

→ A

→ person

→ is

→ throwing

→ a

→ ball

→ <END>

5. Training

During training, the predicted caption is compared with the actual caption using a loss function such as **categorical cross-entropy**. BPTT is used to update the LSTM parameters.

Complete Working

Video → Frames → Pre-trained CNN → Visual Features → LSTM Encoder → LSTM Decoder → Caption

Example:

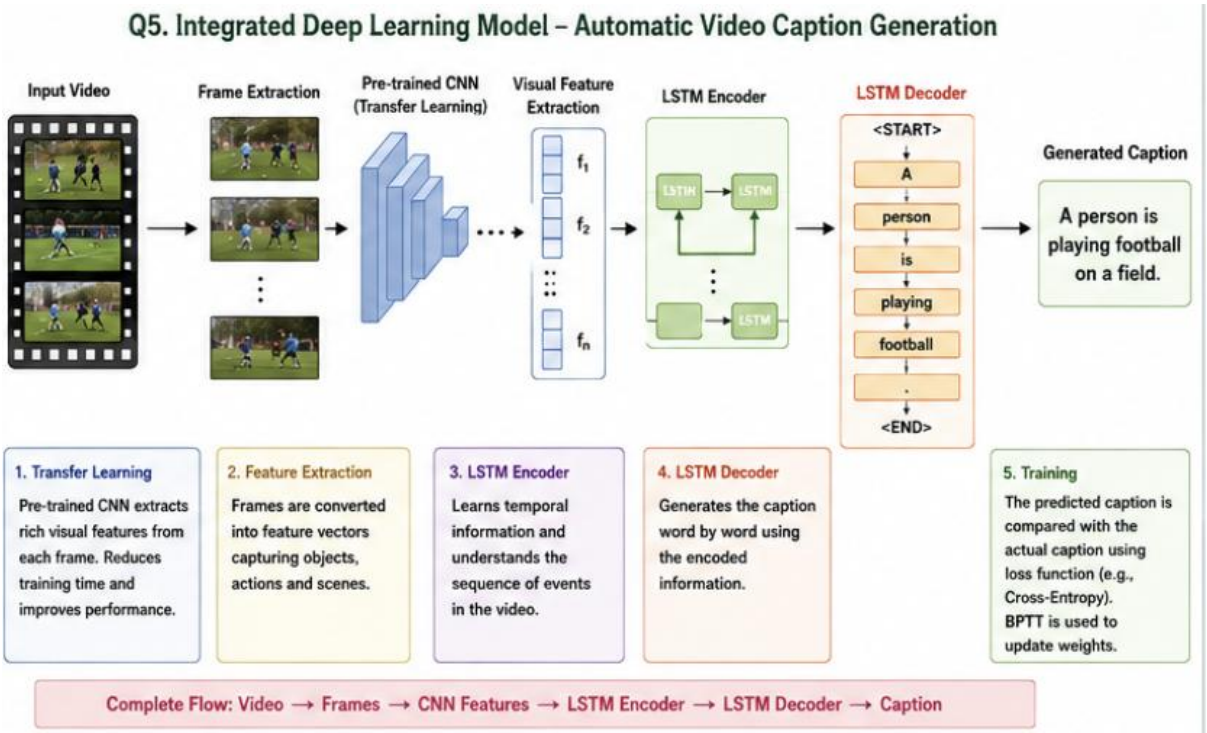
Input video: A person is playing football.

Output:

"A person is playing football on a field."

Advantages

- Transfer learning reduces training requirements.
- CNN extracts powerful visual features.
- LSTM captures temporal relationships.
- Encoder–decoder architecture generates captions sequentially.
- The complete system can produce meaningful descriptions of video content.



Code of Conduct:

I D.Suryakiran/192325057(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases